Spec: runevault

Exported: 8/7/2026, 11:31:34 AM

User:

Create the tasks for runevault

Assistant:

The user wants to generate the tasks for an existing spec. Let me check the current state of the runevault spec.

I have an existing spec. Let me read the config to determine the workflow type and confirm prerequisites.

Both prerequisite documents (requirements and design) exist. This is a design-first feature spec, so I'll generate the task list now.

Sub-Agent: Feature Design First Workflow

Explanation: Generating the tasks document for the existing design-first runevault spec, since both prerequisite documents already exist.

Prompt:

Generate the implementation task list (tasks.md) for the existing "runevault" feature spec. This is a design-first feature workflow. The requirements.md and design.md documents already exist in .kiro/specs/runevault/. Create tasks.md based on the existing design and requirements documents.

Response:

Created tasks.md for the design-first "runevault" feature spec at .kiro/specs/runevault/tasks.md, derived from the existing design.md and requirements.md.

Key decisions:

- No implementation-language prompt was needed: design.md specifies C++ (C++14/17) explicitly (not pseudocode).
- The design contains a full "Correctness Properties" section (21 properties) using rapidcheck, so property-test sub-tasks are included, each marked optional (*), annotated with its Property number and the requirement clauses it validates.
- Config file (.config.kiro) already existed, so it was not recreated.

Structure:

- 20 top-level task groups following the design's layered build order (Geometry → Board/View → Generalized actors → Movement/Combat → Strategy/Visitor patterns → Decorator scrolls → State-based guarded items → Spawner random generation → Level transition → Controller → CLI options → Integration), with 4 checkpoint tasks interspersed.
- Two-level hierarchy with decimal sub-tasks; test sub-tasks marked with * (core implementation never optional).
- Every requirement (1.1–13.3) is referenced across implementation tasks; all 21 correctness properties mapped to dedicated property-test sub-tasks.
- Appended a valid Task Dependency Graph (26 contiguous waves, 0–25) covering all leaf sub-tasks including optional ones, with same-file/dependent tasks separated across waves.

Fixed an initial duplicate sub-task number in section 12 and updated the dependency graph accordingly. Kiro Spec Format diagnostics pass with no issues.

This completes the design-first workflow (artifact creation only). No implementation was performed. The user can begin execution by opening tasks.md and clicking "Start task" next to a task item.

The task list for RuneVault is ready. It breaks the implementation into 20 incremental steps that follow the design's layered build order, from core geometry and the board renderer through actors, combat, the pattern-based systems (Strategy, Visitor, Decorator, State), the random spawning pipeline, and finally the controller and CLI wiring. Optional property-based test sub-tasks are interspersed and traced back to the correctness properties and requirements, with checkpoints along the way to validate progress.

The spec is ready for implementation! You can now start running individual tasks or use the Run All Tasks button to queue up the entire Task List.
